

"OOP Introduction"

- Is A Paradigm or style of code.
- Use the Concept of object to design A Computer Program.
- Its not new $\Rightarrow$ Simula is the first oop Programming language at 1960
- some languages Support oop and some not.
- object is A Package contains Properties and Functions that work together to Produce something in your Application. Functions here called Methods.

why we use oop ?

- You will be Able to create a large and Complex software Architecture in organized ways.
- You will be Able to hide certain Parts of code in your object with Encapsulation to Prevent mess with the code.
- You will be able to create Reusable objects to use in your Application easily with Inheritance.

#148

رقم:

NO: ٢٠٢٦/١١/١٤

ROX

Date:

التاريخ:

Constructor Function Introduction

```

Function User(id, username, salary) {
  this.i = id;
  this.u = username;
  this.s = salary;
}

```

تكويد على
ال object الذي
أنشأت.

Let userOne = new User(100, "ElZero", 500)

كده أنا أنشأت object

three Properties جديد هكون من Values

من طريقة ايل قيسه من

new + Function وأقدر أغير من القيد من
خلال ال Function

*

*

"constructor Function New" Syntax

```
Function User(id, username, salary) {
  this.i = id;
  this.u = username;
  this.s = salary;
}
```

```
Let userOne = new User(100, "ElZero", 5000);
```

بجمل Focus

+ ctrl + dot (.)

مظهر options مختار الاختيار الذي فيه

Function to an ES6 class

ES6 Function لشكلها

instance of يعرفه object

instance من

ex: userOne instanceof User

True

userOne.constructor === User

True

لأن User كلاس

constructor function

"Deal with Properties & Methods"

القامل مع ال Properties و ال Methods الموجودة داخل ال class

```

class User {
  constructor(id, username, salary) {
    // Properties
    this.i = id;
    this.u = username || "Un Known";
    this.s = salary < 6000 ? salary +
      500 : salary;
    this.msg = function () {
      return `Hello ${this.u}`;
    };
  }

  // Methods
  writeMsg () {
    return `Hello ${this.u}`;
  }
}

let userone = new User(1, "John", 5000)

```

Diagram showing the arguments passed to the User constructor:

- id (points to 1)
- username (points to "John")
- salary (points to 5000)

Update Properties And Built In Constructors

```
class User {
  constructor( " , " , " ) {
    this.i = " ";
    " " " "
    " " " "
  }
```

```
  updateName( new Name ) {
```

```
    this.u = new Name ;
```

```
  } update يقوم بجس Method
```

```
  } . username
```

```
let userOne = new User( " , " , " )
```

```
  . Ali
```

```
console.log( userOne.u ); → "Ali"
```

```
userOne.updateName( "osama" );
```

```
console.log( userOne.u ); → "osama";
```

```
let strOne = "Elzero"; → string
```

```
let strTwo = new string( "Elzero" );
```

```
  object
```

```
strOne instanceof string → False
```

```
strTwo instanceof string → True
```

```
strOne.constructor === string → True
```

```
strTwo. " " " " ~ → True
```

class static Properties And Methods

منشأ وقدرأ وصلها عن من

class user { . Class الـ

// static Properties .

static count = 0;

constructor (" , " , ") {

this.i = id;

" " " من كل مرة منشأ

" " " فيها object جديد الـ count

user.count ++ ; - 1

}

// static Methods .

static sayHello () {

return 'Hello From Class ;

}

static countMembers () {

return '{ this.count } members created' ;

}

}

let userOne = new User (" " , " ")

" two

" three

User

User.count / User.sayHello
User.countMembers

Class Inheritance

// Parent class

```
class User {
    id      username
    constructor ( , , ) {
        this.i = id;
        this.u = username;
    }
    sayHello () {
        return 'Hello ${this.u}';
    }
}
```

3 Properties Admin class اللى اسمها Admin هيا

// Drived class
Class Admin extends User {

```
    constructor ( , , ) {
        id      username      Permissions
```

~~super~~ super(id, username)

this.P = Permissions;

Properties Admin class اللى هتكون
الى عاورها ~~من~~ من User و Admin

}

P

```
let adminOne = new Admin( , , );
adminOne.i →
```

adminOne.sayHello() → Hello (u)

"Class Encapsulation"

- Class Fields Are Public By Default.
- Guards the Data Against Illegal Access
- Helps To Achieve the Target without Revealing Its complex Details.
- will Reduce Human Errors.
- Make the app More Flexible And Manageable.
- simplifies The APP.

بوجود أيضاً

Class User {
// Private Property → Private Methods

#e; id username

Constructor(" ", " ", " ") {

this.i = id;

→ salary

this.#e = e salary;

} Class استخدمها داخل

getSalary() {

return parseInt(this.#e);

} ولكن مباشرة أوصل e مباشرة.

→ يخرج على الرقم الموجود {
من (e)

let user one = new User(, ,)

userone.getSalary()

object. Prototype. love = " " " "

۱) کیہ انا حقیقتہً حق ال object

Method تعوّده بطباعة ال string ،

love (leaf)

string.prototype.addDot = Function

11 11 11

3

که اینها هم method. ۱۱

String object
String

dot string کے لغوی معنی

فصل string

object Meta Data And Descriptor Part 1 → Key.

Descriptor

Part

Key.

object.defineProperty(, " " , {

myobject

my object ✓

writable :  true or false.

→ تَمَكُّمٌ من لِإِسْمَاعِيلِيَّةٍ تَقْنِيَرٍ قَيْسَةٍ فِيمَا لَيْدَ .

enumerable: ☐ true or false

↓
 متحرک کی سماجیہ حل looping
 variable:  

← بتکرم من لسانیات کے لیے
 configurable: ☐ ; true or false
 ← بتکرم من لسانیات کے لیے

Property (من الاموال) (True or False)

القيمة $\rightarrow$ Property Value: $\square$

القصة

لادفظ: مشعره اكل Redefined لو
False configurable Configurable

"Object Meta Data Descriptor" #158 Part 2

- Define Multiple Properties.
- check Descriptors.

myobject

syntax:

object.defineProperty(, {

c: {

configurable: true;

value: 3;

}, Property  يقوم بإعطائها خواص

d:

" " "

myobject

object.getOwnPropertyDescriptor(, )

Property object

myobject + (s) 

منظور كل خاصية موجودة
من object